File Security

🧠 First: What is “File Security”

When users upload files, risks include:

- Malware (virus, trojan, ransomware)
- Executable scripts hidden as images
- Large files crashing your server
- Path traversal attacks
- Zip bombs
- Fake MIME types
- Stored XSS (malicious HTML inside files)

So your system must defend against:

1. Invalid file type
2. Oversized file
3. Malware content
4. Dangerous filename
5. Public access before validation

✅ Proper Secure Upload Architecture

User

↓

Backend receives file

↓

Temporary storage (quarantine)

↓

Validation checks

↓

Antivirus scan

↓

If safe → move to permanent storage

If unsafe → delete

🔹 STEP 1 — Upload to Temporary Location (Quarantine)

When backend receives file:

- Save it in a non-public folder
- Example: `/tmp/uploads`
- Do NOT store inside `/public`
- Do NOT store in final cloud bucket yet

This isolates untrusted content.

🔹 STEP 2 — Validate File Size (Backend)

Immediately check:

- Max size (e.g., 10MB)
- Reject if exceeded
- Delete file immediately

Large files can:

- Crash server memory
- Cause denial-of-service attacks
- attacks

🔹 STEP 3 — Validate File Type (Real Validation)

Never trust:

- File extension (.jpg)
- Browser MIME type

Instead:

Check file signature (magic bytes).

Example:

- PDF files start with `%PDF`
- PNG starts with specific binary signature

Use backend libraries that detect real MIME type from content.

Reject:

- .exe
- .bat
- .js
- .sh
- .php
- .html (unless necessary)

Allow only specific safe types.

This is called **whitelisting**.

🔹 STEP 4 — Rename the File

Never store the original filename.

Generate:

- UUID
- Random hash

This prevents path traversal and overwrite attacks.

🔹 STEP 5 — Antivirus / Malware Scanning

This is the most important layer.

Use antivirus engine like:

- ClamAV

How it works:

1. Backend sends file to scanner
2. Scanner checks virus signatures
3. Returns CLEAN or INFECTED

If INFECTED:

- Delete file
- Log event
- Return error to user

If CLEAN:

- Continue

🔹 STEP 6 — Move to Permanent Storage

Only after passing all checks:

Move file to:

- Secure storage folder
- Or cloud storage like:
    - Amazon S3
    - Google Cloud Storage

Store metadata in database:

- fileId
- userId
- uploadDate
- filePath
- fileSize

🔹 STEP 7 — Control File Access

Even after upload:

Do NOT make file public automatically.

Options:

1. Serve files through backend route (protected)
2. Use signed URLs (temporary access)
3. Check user permission before download

Never expose direct folder paths.

🧩 Complete Secure Upload Flow (Step-by-Step Summary)

1. User selects file
2. Backend saves file in quarantine folder
3. Validate:
    - Size
    - Type (magic bytes)
4. Rename file
5. Scan with antivirus
6. If clean → move to storage
7. Save metadata in database
8. Allow controlled access
9. If infected → delete + log

Think in 3 layers:

Layer 1 — Validation (size + type)
Layer 2 — Malware Scan
Layer 3 — Secure Storage + Access Control

If any layer fails → reject